

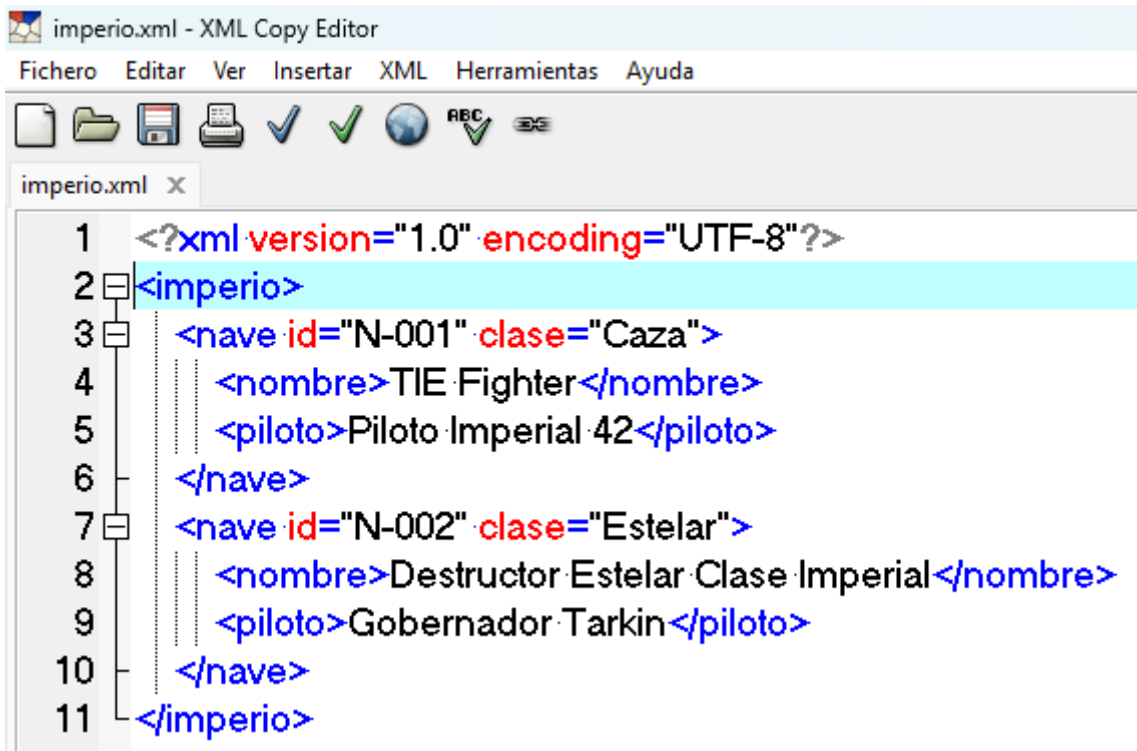
LABORATORIO 1

SISTEMA DE ARCHIVOS ARMADA REBELDE

Sumario

- Configuración del documento base..... 3
- Análisis estructural con Parser DOM..... 4
- Auditoría ligera con Parser SAX..... 5
- Inteligencia de datos mediante binding..... 6

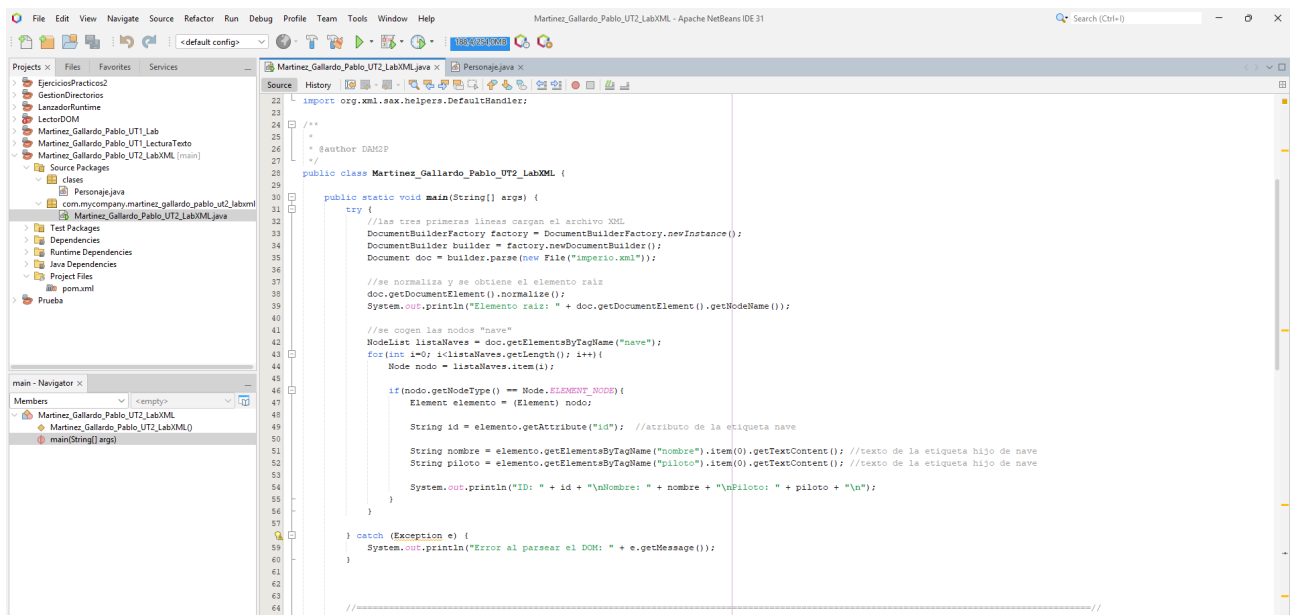
Configuración del documento base



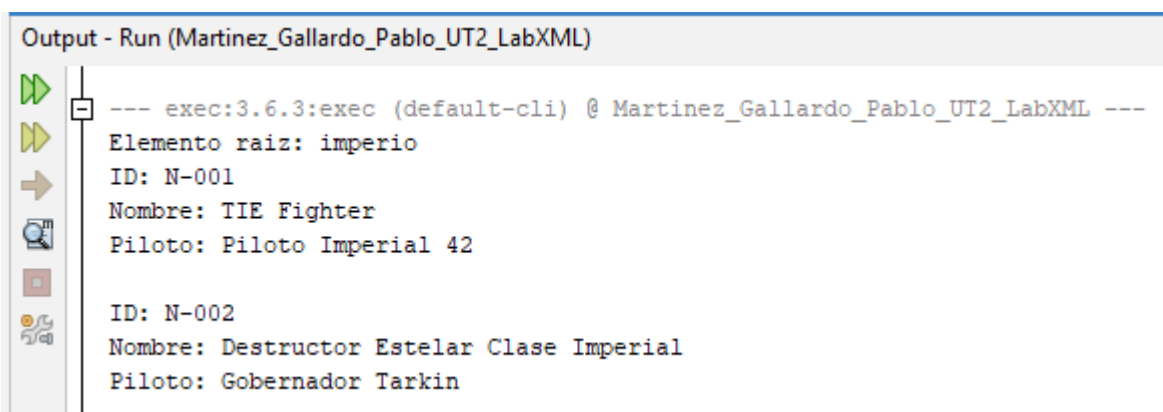
```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <imperio>
3   <nave id="N-001" clase="Caza">
4     <nombre>TIE Fighter</nombre>
5     <piloto>Piloto Imperial 42</piloto>
6   </nave>
7   <nave id="N-002" clase="Estelar">
8     <nombre>Destructor Estelar Clase Imperial</nombre>
9     <piloto>Gobernador Tarkin</piloto>
10  </nave>
11 </imperio>
```

Se creó el documento XML para poder hacer los demás ejercicios.

Análisis estructural con Parser DOM



```
22 import org.xml.sax.helpers.DefaultHandler;
23
24 /**
25  *
26  * @author DAMOP
27  */
28 public class Martinez_Gallardo_Pablo_UT2_LabXML {
29
30     public static void main(String[] args) {
31         try {
32             //las tres primeras lineas cargan el archivo XML
33             DocumentBuilderFactory factory = DocumentBuilderFactory.newInstance();
34             DocumentBuilder builder = factory.newDocumentBuilder();
35             Document doc = builder.parse(new File("imperio.xml"));
36
37             //se normaliza y se obtiene el elemento raiz
38             doc.getDocumentElement().normalize();
39             System.out.println("Elemento raiz: " + doc.getDocumentElement().getNodeName());
40
41             //se cogen las nodos "nave"
42             NodeList listaNaves = doc.getElementsByTagName("nave");
43             for (int i=0; i<listaNaves.getLength(); i++){
44                 Node nodo = listaNaves.item(i);
45
46                 if (nodo.getNodeType() == Node.ELEMENT_NODE) {
47                     Element elemento = (Element) nodo;
48
49                     String id = elemento.getAttribute("id"); //atributo de la etiqueta nave
50
51                     String nombre = elemento.getElementsByTagName("nombre").item(0).getTextContent(); //texto de la etiqueta hijo de nave
52                     String piloto = elemento.getElementsByTagName("piloto").item(0).getTextContent(); //texto de la etiqueta hijo de nave
53
54                     System.out.println("ID: " + id + "\nNombre: " + nombre + "\nPiloto: " + piloto + "\n");
55                 }
56             }
57         } catch (Exception e) {
58             System.out.println("Error al parsear el DOM: " + e.getMessage());
59         }
60
61
62
63
64 //=====
```

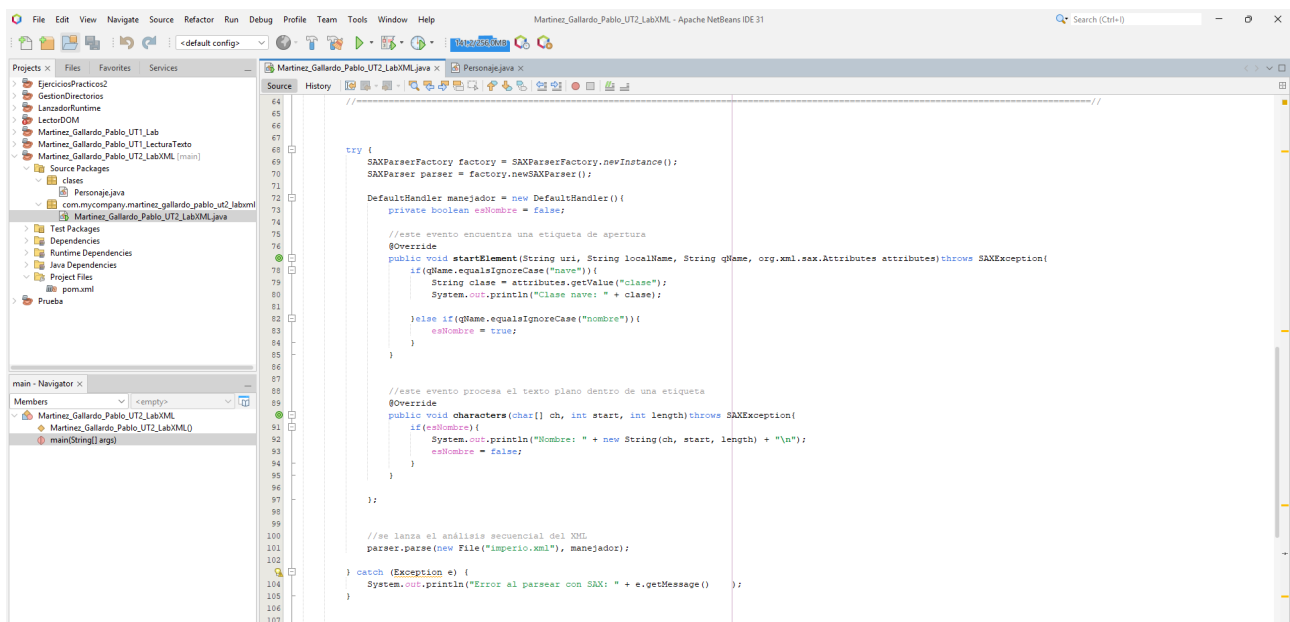


```
Output - Run (Martinez_Gallardo_Pablo_UT2_LabXML)
--- exec:3.6.3:exec (default-cli) @ Martinez_Gallardo_Pablo_UT2_LabXML ---
Elemento raiz: imperio
ID: N-001
Nombre: TIE Fighter
Piloto: Piloto Imperial 42

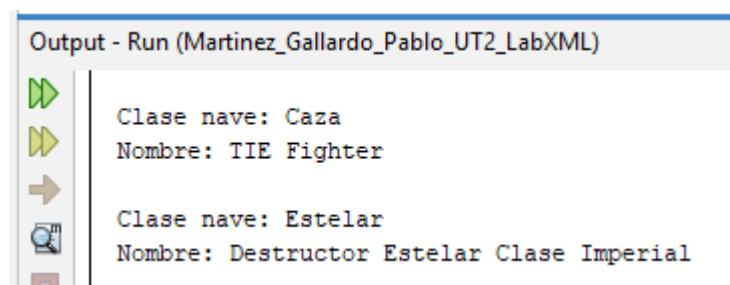
ID: N-002
Nombre: Destructor Estelar Clase Imperial
Piloto: Gobernador Tarkin
```

Empezamos utilizando el `DocumentBuilderFactory` y el `DocumentBuilder` para poder cargar el archivo XML en la variable “doc” y después se normaliza. Después con el `NodeList` añadimos todas las etiquetas “nave” del archivo XML para después recorrerlas todas con un bucle, guardar el atributo del id, y las etiquetas del nombre y del piloto para imprimir por pantalla todas las naves con sus datos correspondientes.

Auditoría ligera con Parser SAX



```
64
65
66
67
68 try {
69     SAXParserFactory factory = SAXParserFactory.newInstance();
70     SAXParser parser = factory.newSAXParser();
71
72     DefaultHandler manejador = new DefaultHandler() {
73         private boolean esNombre = false;
74
75         //este evento encuentra una etiqueta de apertura
76         @Override
77         public void startElement(String uri, String localName, String qName, org.xml.sax.Attributes attributes) throws SAXException {
78             if (qName.equalsIgnoreCase("nave")) {
79                 String clase = attributes.getValue("clase");
80                 System.out.println("Clase nave: " + clase);
81
82                 //este evento procesa el texto plano dentro de una etiqueta
83                 @Override
84                 public void characters(char[] ch, int start, int length) throws SAXException {
85                     if (!esNombre) {
86                         System.out.println("Nombre: " + new String(ch, start, length) + "\n");
87                         esNombre = true;
88                     }
89                 }
90             }
91         }
92     };
93
94     //se lanza el análisis secuencial del XML
95     parser.parse(new File("imperio.xml"), manejador);
96
97     catch (Exception e) {
98         System.out.println("Error al parsear con SAX: " + e.getMessage());
99     }
```



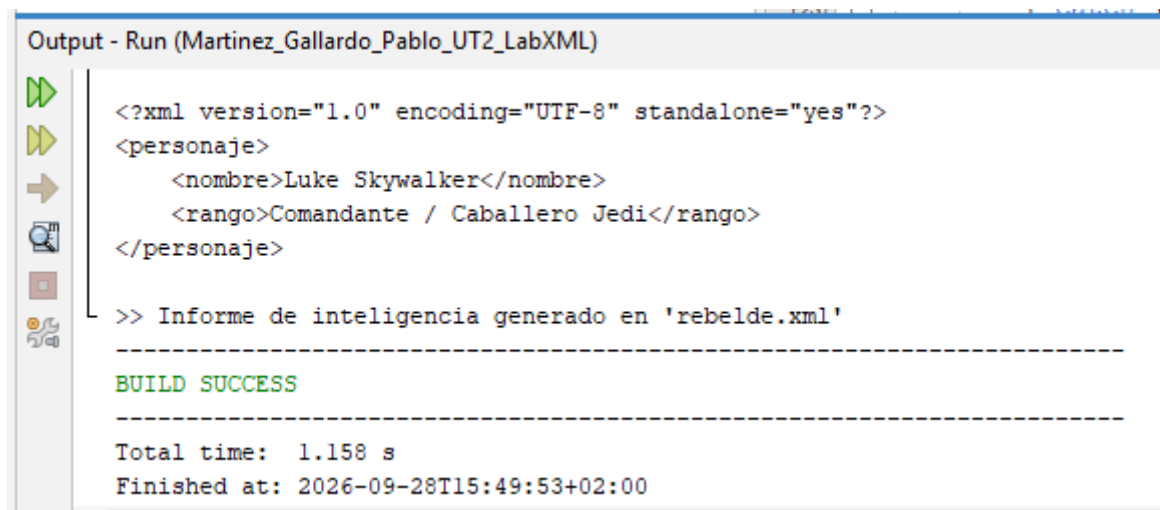
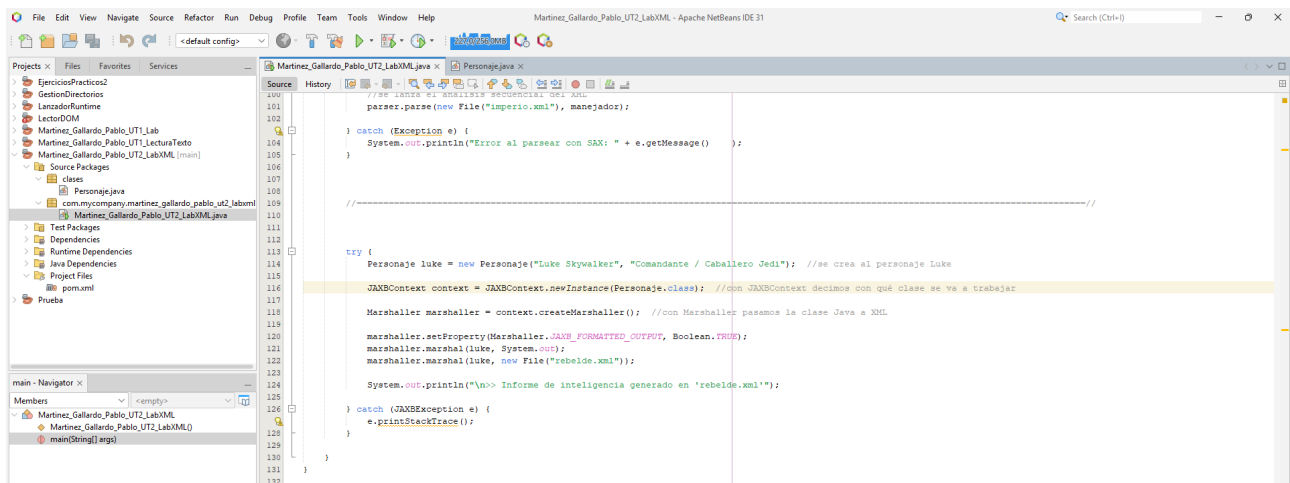
```
Output - Run (Martinez_Gallardo_Pablo_UT2_LabXML)

Clase nave: Caza
Nombre: TIE Fighter

Clase nave: Estelar
Nombre: Destructor Estelar Clase Imperial
```

Con SAXParserFactory y SAXParser se deja todo preparado para después leer el archivo XML, pero antes se tienen que comentar dos funciones que tiene leer el archivo XML, utilizando un manejador con DefaultHandler. Primero está la función startElement(), que se encarga de encontrar una etiqueta de apertura (en este caso las etiqueta nave) y también almacenar el atributo de la etiqueta nave que necesitamos. Después, la función characters procesa el texto plano dentro de la etiqueta nave, que en este caso es el nombre de la nave. Para terminar, utilizamos el parser para pasarle el archivo XML que queremos leer y el manejador para que nos dé los datos que queremos.

Inteligencia de datos mediante binding



Aquí vamos a pasar una clase de Java y convertirlo en un archivo XML. Primero creamos el objeto que queremos pasar y con JAXBContext le decimos qué clase vamos a decirle que tiene que pasar. Después declaramos un marshaller que será el encargado de pasar la clase Java a XML, le pasamos el objeto creado y el nombre del archivo que queremos que se llame.